From prototype to operated service

Build with GitHub Copilot.
Run, evaluate and improve with Foundry.

1. **BUILD**
   One useful agent
2. **PROVE**
   Evidence before release
3. **IMPROVE**
   Learn from operations

Sridhar Arrabelly | 50 minutes + Q&A

A convincing answer is only the beginning

The harder question: can we explain, release and keep improving it?

1 Useful

It solves a real customer task.

Not just a fluent response.

2 Provable

We can inspect behaviour and measure regressions.

3 Operable

Every release has evidence.

Operations teaches DEV.

Our question today: what evidence earns the next release?

Meet Volt & Co

A fictional electronics assistant that recommends the cheapest product that works.

"A charger for my Volt Book 14. Under EUR 50."

Six synthetic products.

No purchases. No customer data.

1. **Find** — Catalog facts and prices
2. **Check** — Device compatibility
3. **Support** — Batch-specific recall status

Boundary: missing evidence is not permission to claim certainty.

Two loops. Two evidence bridges.

The inner loop proves a candidate. The outer loop releases and learns.

INNER LOOP | DEV

Build + connect

Run + trace

Evaluate + improve



candidate

+ evidence

OUTER LOOP | SERVING

Gate + release

Observe + investigate

Review + feed back

Observed interaction > human review > new regression case > next candidate

Build with GitHub. Operate in Foundry.

GitHub Copilot develops the code. The Copilot SDK is the application's runtime harness.

1 GitHub

Code + Copilot
Tests + pull requests
Actions + evidence

2 Copilot SDK

Constrained harness
Three read-only tools
Foundry model via Entra

3 Foundry

Hosted agent + endpoint
Evaluation + traces
Operational review

We own the business logic. Foundry manages the hosting around it.

A useful answer, with an inspectable path

Find the cheapest charger for my Volt Book 14 under EUR 50.

1. Check the recommendation
2. Inspect the tool evidence
3. Follow the request trace

Live execution where ready. Recorded evidence is labelled and versioned.

A good demo is not a quality baseline

Use the same reviewed cases to compare candidates.

Trust by demo

"It looked right when I tried it."

One execution, unknown coverage.

Trust by evidence

Named cases. Expected behaviour.

Results tied to the exact candidate.

A green score on a small dataset is evidence, not a guarantee.

Three checks, three different jobs

Do not ask one magic score to do everything.

1 Deterministic

Correct product and batch

Required tool evidence

Missing-input contract

2 Model-judged

Native Foundry scorecard

Response coherence

A signal, not domain truth

3 Human review

Expected behaviour

Ambiguous risk decisions

Release acceptance

Useful + correct enough for the agreed cases + reviewable.

Change one thing. Measure what changed.

Baseline > evidence > correction

Same questions. Same assertions.

1. Open a failing case
2. Inspect the code change
3. Compare the candidate

Live execution where ready. Recorded evidence is labelled and versioned.

Release the evidence, not just the prompt

A passing check is meaningful only when the release path enforces it.

1 Verify

Code tests

Reviewed cases

Candidate identity

2 Decide

Required criteria

Human acceptance

Recorded evidence

3 Release

Pinned version

Known configuration

Rollback reference

Foundry supplies capabilities. We configure the release policy.

A release we can explain

Source revision + agent version
+ dataset hash + scorecard

1. Show BLOCK and PASS
2. Inspect the release record
3. Invoke the pinned version

Live execution where ready. Recorded evidence is labelled and versioned.

Operations teaches development what to test

An observation becomes a regression case only after review.

1 Observe

An ambiguous recall enquiry

2 Review

Decide the expected behaviour

3 Add a test

Keep trace provenance

4 Improve

Prove the next candidate

Observed output is evidence. It is not automatically a label.

Close the loop

I have a Volt Charge 65.

Is it safe?

1. Inspect the missing fact
2. Approve one regression row
3. Recheck the corrected version

Live execution where ready. Recorded evidence is labelled and versioned.

Four things to take away

1. **Build for a customer task**
   One use case makes the architecture meaningful.
2. **Prove behaviour, not just wording**
   Inspect the path and compare reviewed cases.
3. **Make release policy explicit**
   A score matters when a gate uses it.
4. **Turn observations into tests**
   Human-reviewed feedback strengthens the next release.

Your questions. Your operating model.

The right next step is a bounded pilot with an agreed evidence contract.

Outcome

What customer task must improve?

Boundary

What must never happen?

Ownership

Who can approve and operate a release?